

第8章 查找

目录

第一节 查找概述与查找表

第二节 查找算法效率的衡量

第三节 静态表查找

第四节 动态表查找

第一节　查找概述与查找表

- 1.1查找的定义
- 1.2查找结果的表现形式

什么是查找



查找是数据结构的一种基本操作，就是在由一组记录组成的集合中寻找属性值符合特定条件的数据元素。若集合中存在符合条件的记录，则查找成功，否则查找失败。查找条件由包含指定关键字的数据元素给出。

查找结果的表现形式

根据不同的应用需求，查找结果有以下表示形式：



如果判断数据结构是否包含某个特定元素，则查找结果为是、否两个状态。

如果根据关键字查找以获得特定元素的其他属性，则查找结果为特定数据元素。



如果数据结构中含有多个关键字值相同的数据元素，需要确定返回首次出现的元素或者是返回数据元素集合。

如果查找不成功，返回相应的信息。

查找表



查找表是一种以同一类型的记录构成的集合为逻辑结构、以查找为核心运算的灵活的数据结构。

查找表分为静态查找表和动态查找表两种。静态查找表是指对表的操作不包括对表的修改的表；动态查找表是指对表的操作包括对表中的记录进行插入和删除的表。

- 建表
- 查找
- 读表
- 删除

第二节　查找算法效率的衡量

平均查找长度

•查找的主要操作是关键字的比较，所以衡量一个查找算法效率优劣的标准是比较次数的期望值。给定值与关键字值的比较次数的期望值也称为平均查找长度(Average Search Length)，记为ASL。

•对于一个含有n个记录的查找表，查找成功时的平均查找长度如下：

$$\mathrm{ASL} = \sum_{i=0}^{n-1} p_i c_i$$

•其中，p_i是查找第i条记录的概率，c_i是查找第i条记录时关键字值和给定值比较的次数。



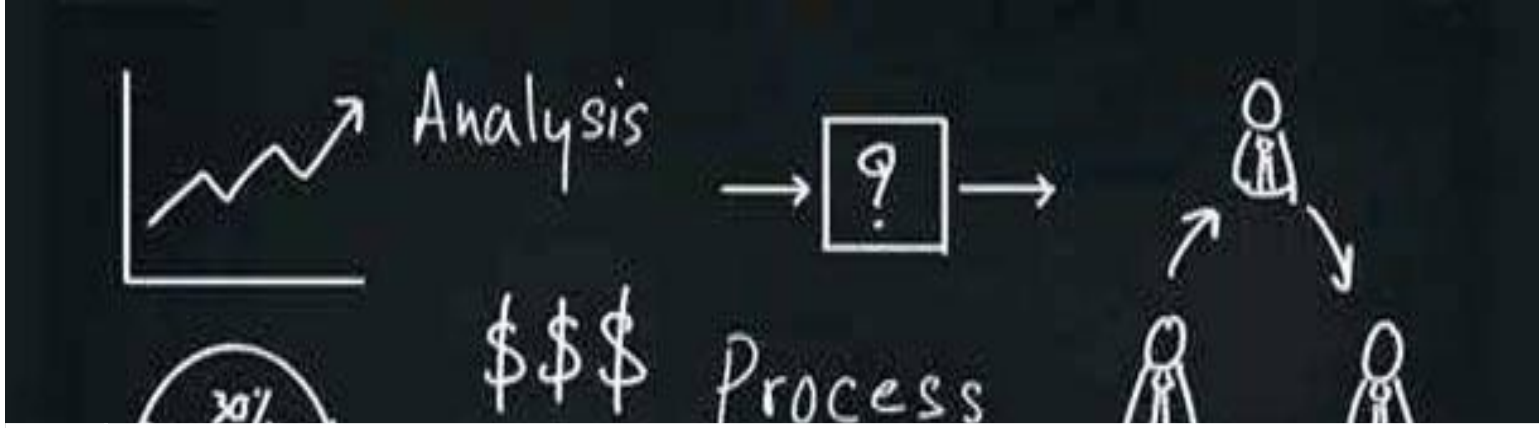
01 02 03 04 GOAL

第三节 静态表查找

- 顺序查找
- 二分查找
- 分块查找

3.1 顺序查找

顺序查找是指从顺序表的一端开始依次将每一个数据元素的关键字值与给定值进行比较，若某个数据元素的关键字值和给定值相等，则查找成功，否则查找失败。顺序查找又叫线性查找。




```
1  def seqSearch(self,key):
2          for i in range(self.len):
3                  if self.list[i].key == key:
4                          return i # 返回关键字值与给性值相等的数据元素的标
5          return -1
```



3.1 算法性能分析

假设查找每个数据元素的概率相等，对于一个长度为n的顺序表，其平均查找长度如下：

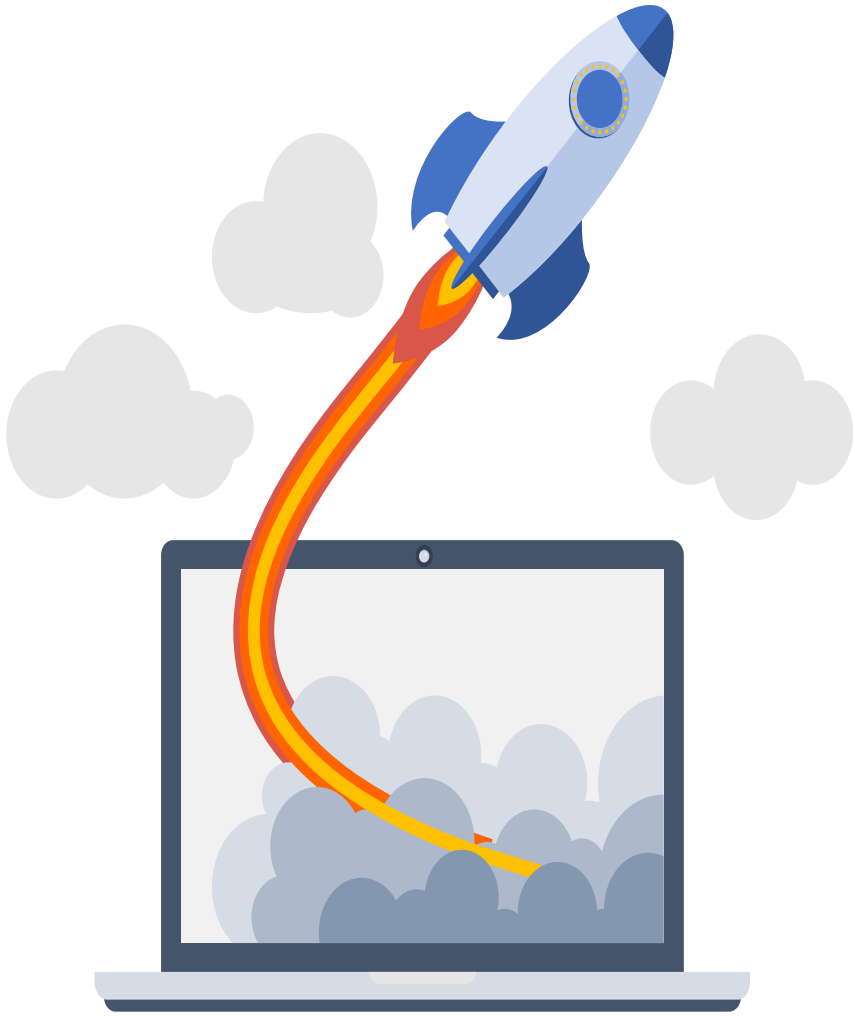
$$\mathrm{ASL}=\sum_{i=0}^{n-1}p_i c_i=\frac{1}{n}\sum_{i=0}^{n-1}(i+1)=\frac{n+1}{2}$$

若查找失败，关键字比较次数为n，因此顺序查找的时间复杂度为O(n)。

3.2 二分查找

1. 二分查找算法的实现

二分查找是对有序表进行的查找。通常假定有序表按关键字值从小到大有序排列，二分查找首先取整个表的中间数据元素的关键字值和给定值进行比较，若相等，则查找成功；若给定值小于该元素的关键字值，则在左子表中重复上述步骤；若给定值大于该元素的关键字值，则在右子表中重复上述步骤，直到找到关键字值为key的记录或子表长度为0。二分查找又叫折半查找。



Python实现

```
1  def binarySearch(self,key):
2          if self.len>0:
3                  # 查找表的上界与下界
4                  low = 0
5                  high = self.len-1
6                  while low<=high:
7                          mid = (low+high)//2 # 取中间元素位置
8                          if self.list[mid].key==key:
9                                  return mid
10                         elif self.list[mid].key<key: # 查找范围为后半部分
11                                 low = mid + 1
12                         else: # 查找范围为前半部分
13                                 high = mid - 1
14         return -1
```

3.2 算法性能分析

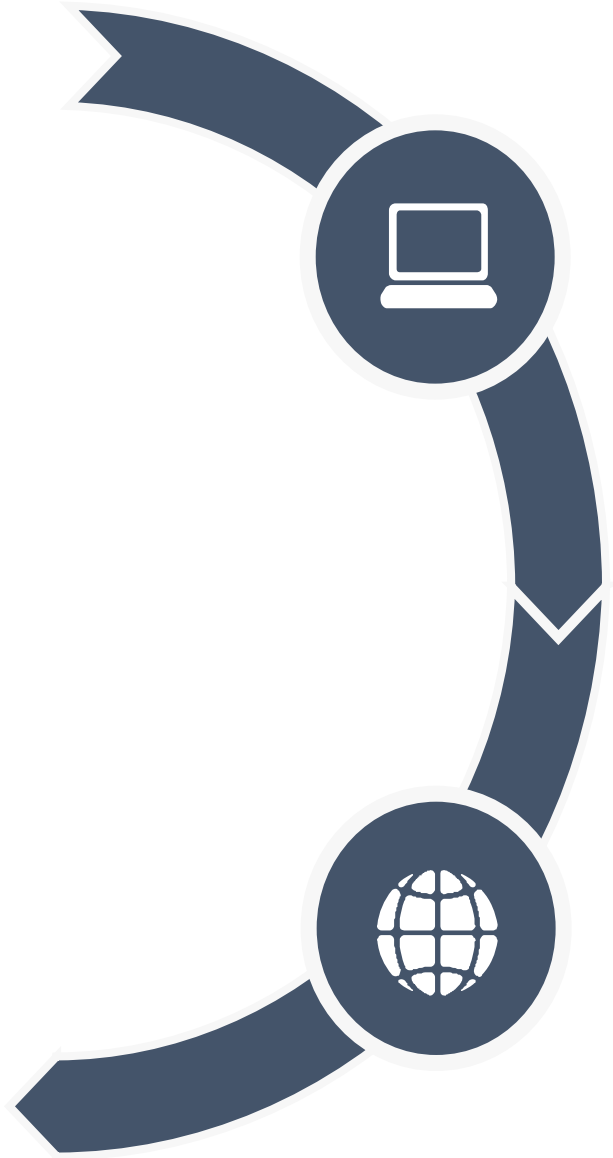
假设查找每个数据元素的概率相等，对于一个长度为n=2k-1的有序表，线性表最多被平分k=log2(n+1)次即可完成查找。又因为在i次查找中可以找到的元素个数为2i-1个，所以其平均查找长度如下：

$$\mathrm{ASL}=\sum_{i=0}^{k}p_i c_i=\frac{1}{n}\sum_{i=0}^{k}(i\times 2^{i-1})=\log_2(n+1)-1+\frac{1}{n}\log_2(n+1)\approx\log_2(n+1)-1$$

因此，查找的时间复杂度为O(log2n)。



3.3 分块查找

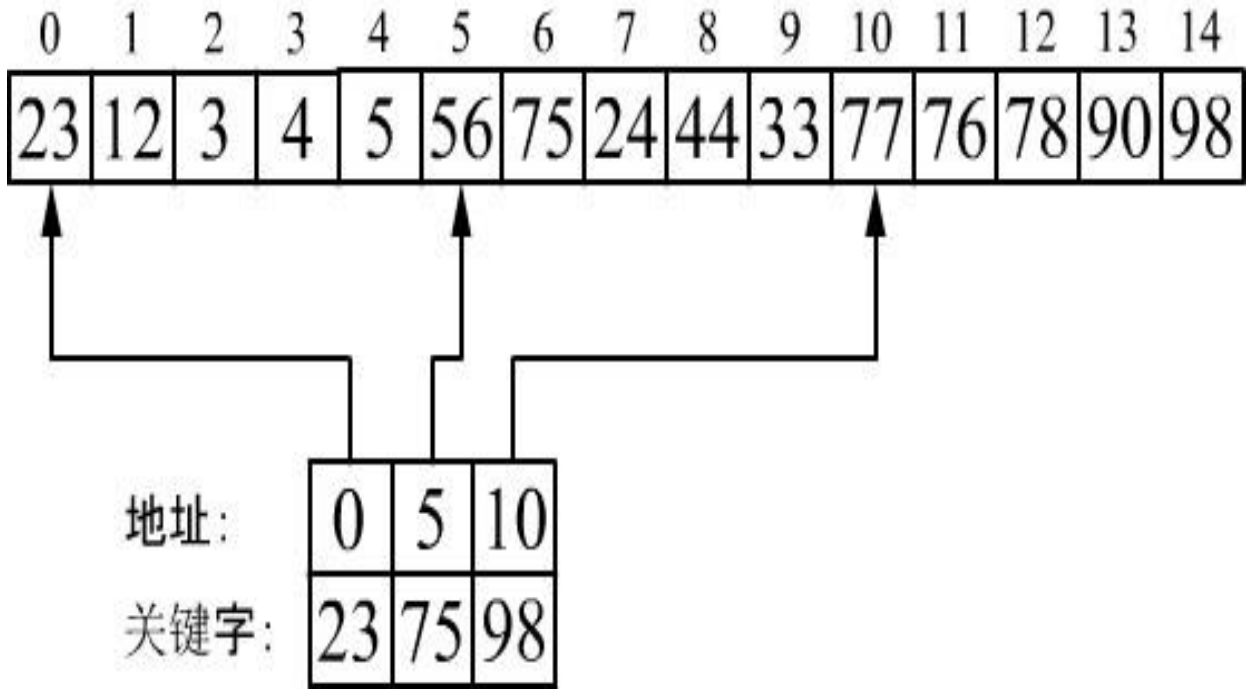


分块查找是将线性表分为若干块，块之间是有序的，块中的元素不一定有序，将每块中最大的关键字值按块的顺序建立索引顺序表，在查找时首先通过索引顺序表确定待查找元素可能所在的块，然后在块中寻找该元素。

索引顺序表是有序表，可以采用顺序查找或者二分查找； 块中元素无序，必须采用顺序查找。

3.3 分块查找

假设线性表中数据元素的关键字为{23,12,3,4,5,56,75,24,44,33,77,76,78,90,98},有15个结点，被分为3块，则要求每一块中的最大关键字值小于后一块中的最小关键字值，分块有序表的索引存储表如图所示。



3.3 算法性能分析

由于分块查找是顺序查找和二分查找的结合，因此分块查找的平均查找长度为查找索引表确定元素所在块的平均查找长度Lb加上在块中查找元素的平均查找长度Lc。

一般将长度为n的线性表均匀分成b块，每块中含有s个元素，即b= 。假定查找每个元素的概率相同，若使用顺序查找确定元素所在的块，则分块查找的平均查找长度如下：

$$\mathrm{ASL} = L_b + L_c = \frac{1}{b}\sum_{i=0}^{b-1}(i+1) + \frac{1}{s}\sum_{i=0}^{s-1}(i+1) = \frac{1}{2}\left(\frac{n}{s}+s\right)+1$$

若使用二分查找确定元素所在的块，则分块查找的平均查找长度如下：

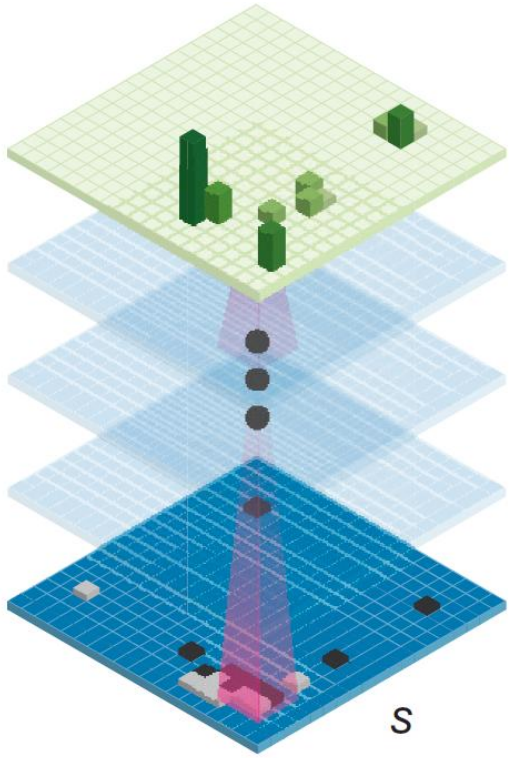
$$\mathrm{ASL} \approx \log_2\left(\frac{n}{s}+1\right)+\frac{s}{2}$$



第四节 动态表查询

- 二叉排序树查找
- 平衡二叉树
- B^-树和B^+树
- 哈希表查找
- 哈希函数
- 解决冲突的方法

4. 动态表查询



- 动态查找表是指对表的操作包括对表的修改的表，即表结构本身实际是在查找过程中动态生成的。动态查找表有多种不同的实现方法，本节中只讨论在各种树形结构上查找的实现方法。

4.1 二叉排序树查找



1、二叉排序树的概念

二叉排序树是具有下列性质的二叉树。

(1) 若右子树非空，则右子树上所有结点的值均大于根结点的值。

(2) 若左子树非空，则左子树上所有结点的值均小于根结点的值。

(3) 左、右子树也为二叉排序树。

二叉排序树可以为空树，其结构如图所示

4.1 二叉排序树查找

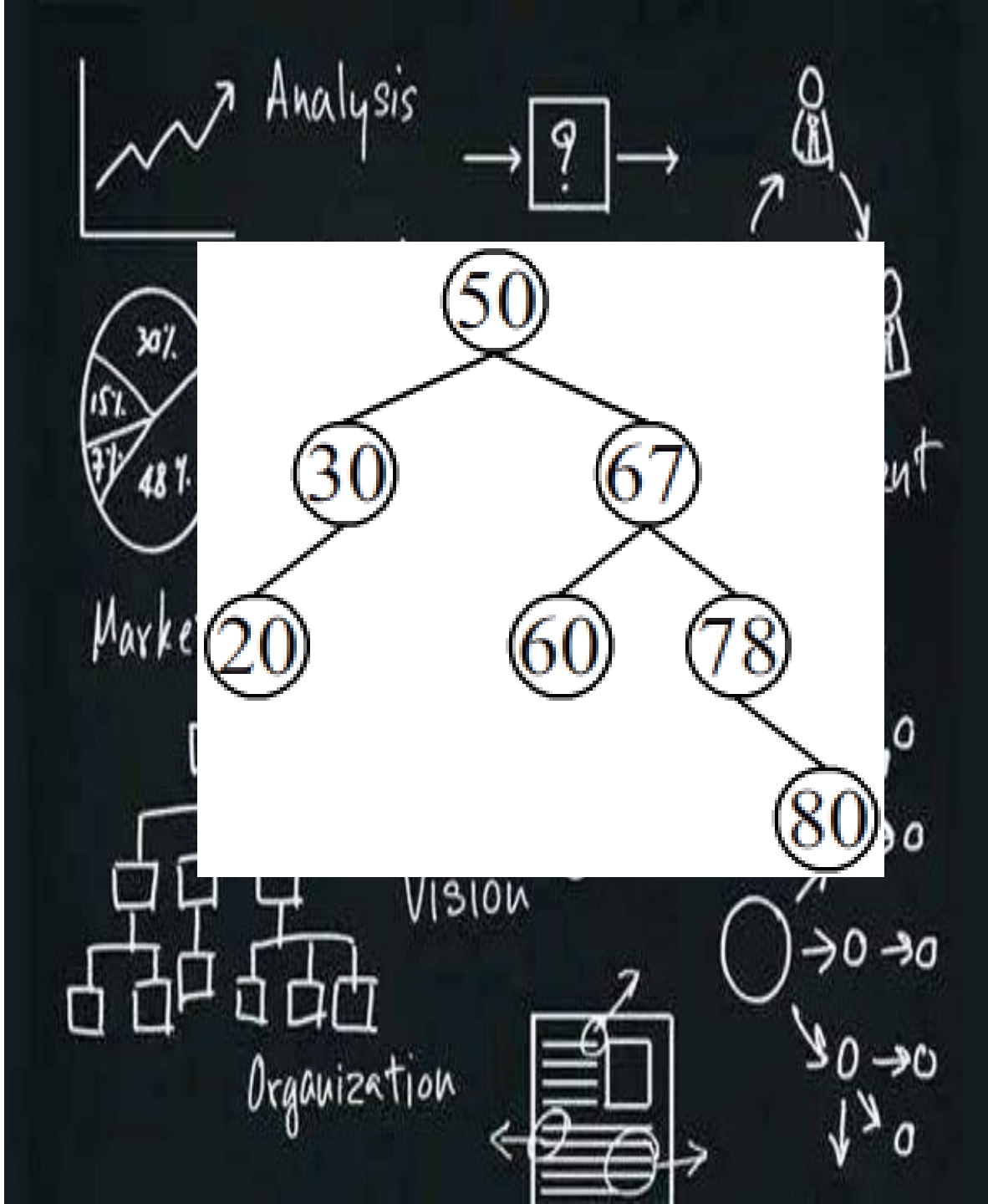
二叉排序树查找过程的主要步骤如下。

(1) 若查找树为空，则查找失败。

(2) 若查找树非空，且给定值key等于根结点的关键字值，查找成功。

(3) 若查找树非空，且给定值key小于根结点的关键字值，在根结点的左子树上进行查找过程。

(4) 若查找树非空，且给定值key大于根结点的关键字值，在根结点的右子树上进行查找过程。



Python实现

二叉排序树的节点算法定义

```
1  class BiTreeNode(object):
2          def __init__(self, key, data, lchild=None, rchild=None):
3                  self.key = key # 结点关键字值
4                  self.data = data # 结点的数据值
5                  self.lchild = lchild # 结点的左孩子
6                  self.rchild = rchild # 结点的右孩子
```

二叉排序树的类结构算法定义

```
1  class BSTree(object):
2          def __init__(self,root=None):
3                  self.root = root # 树的根节点
```

查找算法Python实现

二叉排序树的查找算法

```
1  def search(self,key):
2          return self.searchBST(key,self.root)
3
4  def searchBST(self,key,p):
5          if p is None: # 查找树为空，查找失败
6                  return None
7          if key==p.key: # 查找成功
8                  return p.data
9          elif key<p.key: # 在左子树中查找
10                 return self.searchBST(key,p.lchild)
11         else: # 在右子树中查找
12                 return self.searchBST(key,p.rchild)
```

插入算法Python实现

二叉排序树插入算法的实现

在向二叉排序树中插入一个结点时首先对二叉排序树进行查找，若查找成功，则结点已存在，不需要插入； 若查找失败，再将新结点作为叶子结点插入到二叉排序树中。

```
1  def insert(self,key,data):
2          p = BiTreeNode(key,data) # 为元素建立结点
3          if self.root is None: # 若根节点为空，建立新根节点
4                  self.root = p
5          else:
6                  self.insertBST(self.root,p)
7
8  def insertBST(self,r,p):
9          if r.key<p.key: # 查找右子树
10                 if r.rchild is None:
11                         r.rchild = p
12                 else:
13                         self.insertBST(r.rchild,p)
14         else: # 查找左子树
15                 if r.lchild is None:
16                         r.lchild = p
17                 else:
18                         self.insertBST(r.lchild,p)
```

二叉排序树删除算法的实现

在二叉排序树中删除一个元素要保证删除后的树仍然是二叉排序树，分为3种情况进行讨论。

(1) 若待删除的结点是叶子结点，可直接删除。

(2) 若待删除的结点只有左子树或右子树，则将左子树或右子树的根结点代替被删除结点的位置。

(3) 若待删除的结点有左、右两棵子树，在中序遍历下则将待删除结点的前驱结点或后继结点代替被删除结点的位置，并将该结点删除。



Python实现

```
def remove(self,key):
        # 删除关键字为key的结点
        self.removeBST(key,self.root,None)

def removeBST(self,key,p,parent):
        if p is None: # 树空，直接返回
                return
        if p.key>key: # 在左子树中删除
                self.removeBST(key,p.lchild,p)
        elif p.key<key: # 在右子树中删除
                self.removeBST(key,p.rchild,p)
        elif p.lchild is not None and p.rchild is not None: # 删除
                inNext = p.rchild
                while inNext.lchild is not None:
                        inNext = inNext.lchild
                p.data = inNext.data
                p.key = inNext.key
                self.removeBST(p.key,p.rchild,p)
        else: # 只有一棵子树或者没有子树
                if parent is None:
                        if p.lchild is not None:
                                self.root = p.lchild
                        else:
                                self.root = p.rchild
                        return
                if p==parent.lchild:
                        if p.lchild is not None:
                                parent.lchild = p.lchild
                        else:
                                parent.lchild = p.rchild
                elif p==parent.rchild:
                        if p.lchild is not None:
                                parent.rchild = p.lchild
                        else:
                                parent.rchild = p.rchild
```

4.2 平衡二叉树

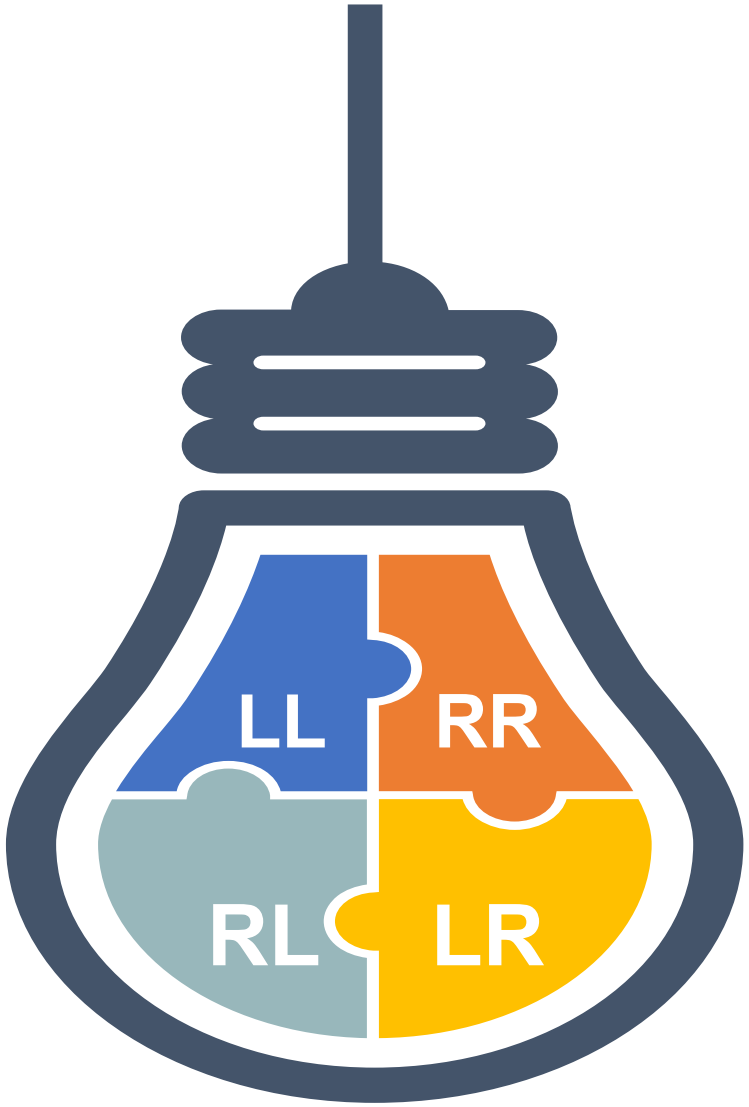


1. 平衡二叉树的概念

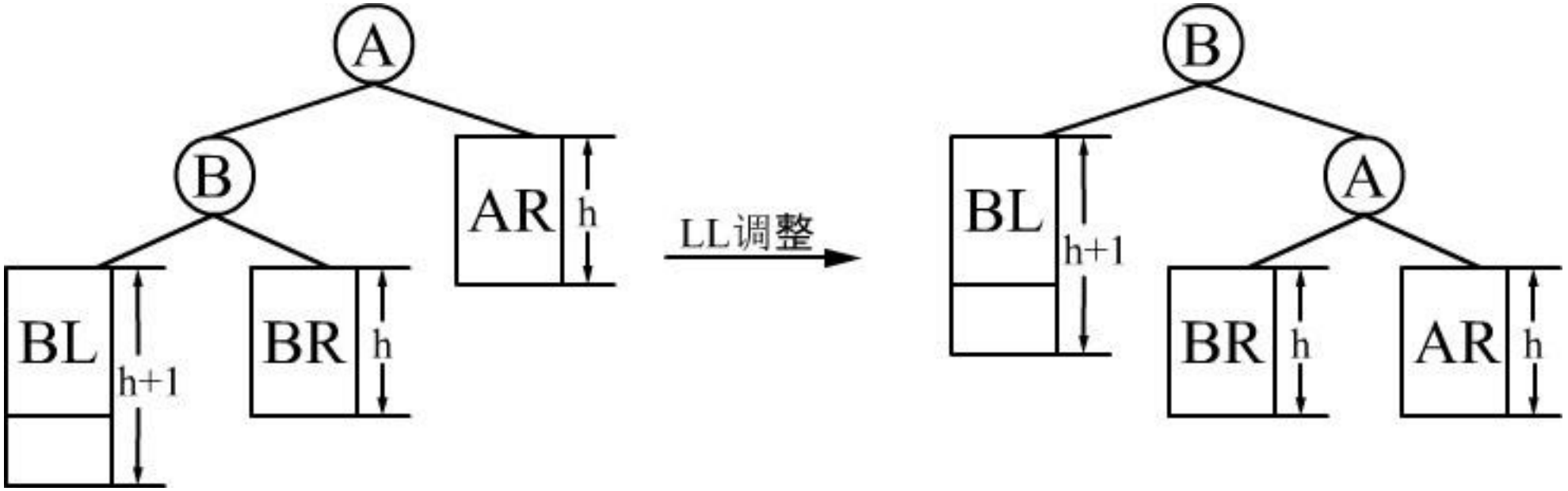
平衡二叉树是左、右子树深度之差的绝对值小于2并且左、右子树均为平衡二叉树的树。平衡二叉树又叫AVL树，可以为空，其某个结点的左子树深度与右子树深度之差称为该结点的平衡因子或平衡度。

4.2 平衡二叉树的实现

在平衡二叉树上删除或插入结点后可能会使二叉树失去平衡。对非平衡二叉树的调整可依据失去平衡的原因分为4种情况进行，分别是LL型、RR型、LR型、RL型(假设在平衡二叉树上因插入新结点而失去平衡的最小子树的根结点为A)。



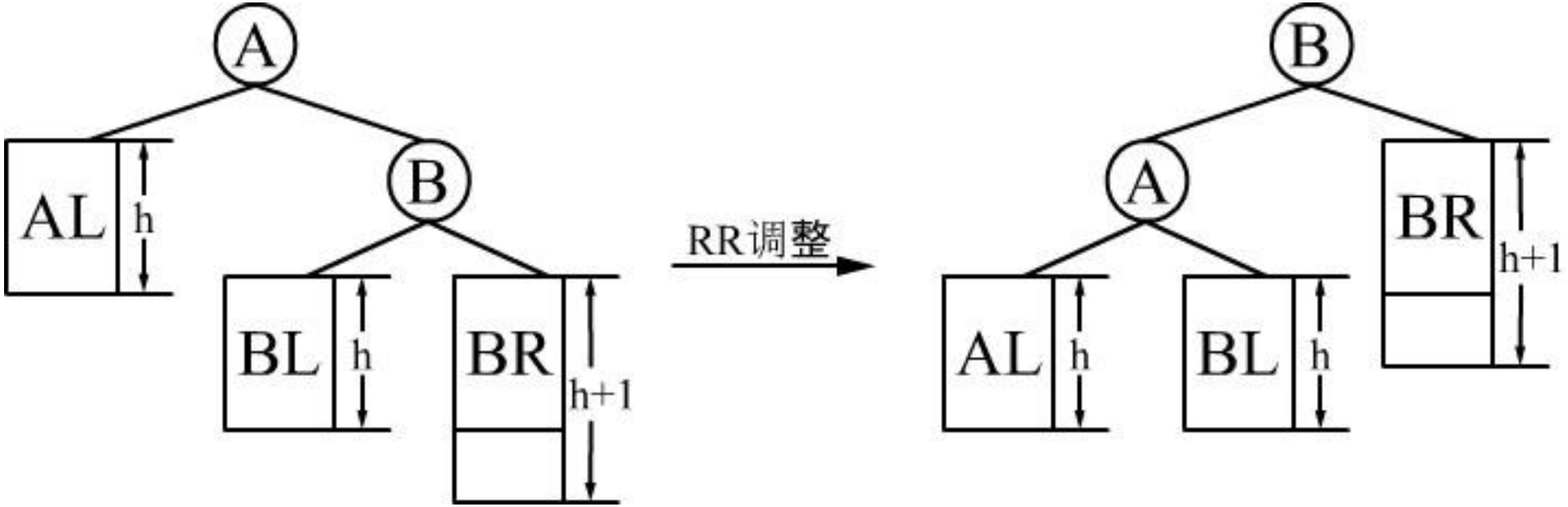
LL型平衡旋转(单向右旋)



原因： 在A的左孩子的左子树上插入新结点，使A的平衡度由1变为2，以A为根的子树失去平衡。

调整： 提升A的左孩子B为新子树的根结点，A为B的右孩子，同时将B的右子树BR调整为A的左子树，如图所示。

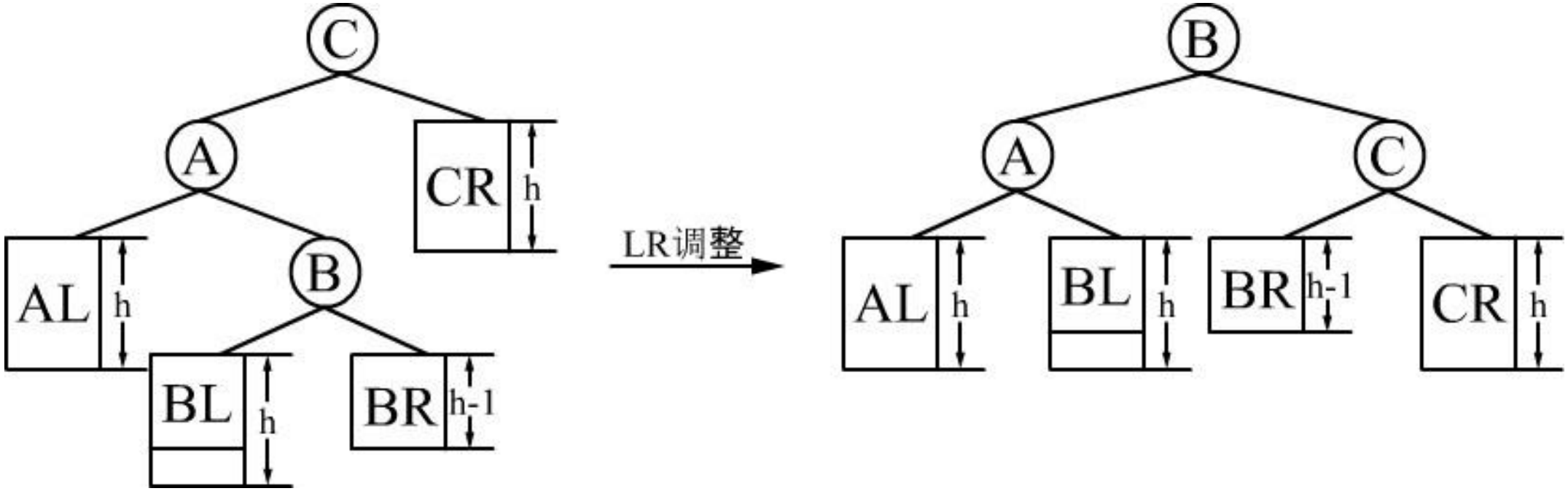
RR型平衡旋转(单向左旋)



原因： 在A的右孩子的右子树上插入新结点，使A的平衡度由-1变为-2，以A为根的子树失去平衡。

调整： 提升A的右孩子B为新子树的根结点，A为B的左孩子，同时将B的左子树BL调整为A的右子树，如图所示。

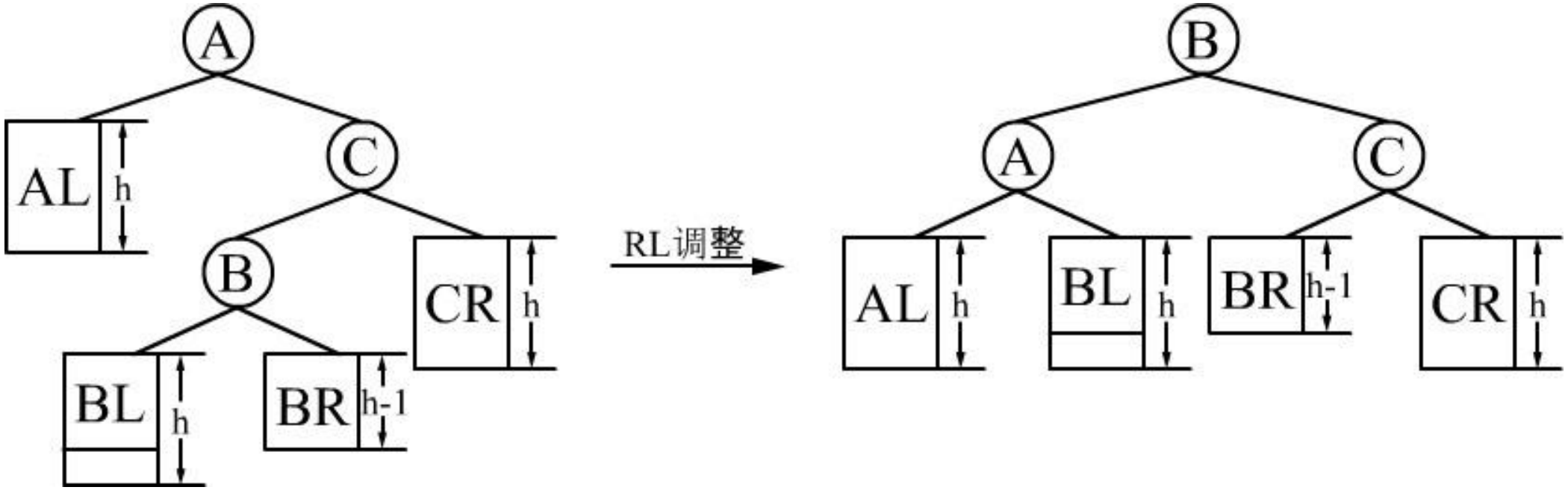
LR型平衡旋转(先左旋后右旋)



原因： 在C的左孩子的右子树上插入新结点，使C的平衡度由1变为2，以C为根的子树失去平衡。

调整： 提升C的左孩子A的右孩子B为新子树的根结点，C为B的右孩子，A为B的左孩子，将B的左子树BL调整为A的右子树，将B的右子树BR调整为C的左子树，如图所示。

RL型平衡旋转(先右旋后左旋



原因： 在A的右孩子的左子树上插入新结点，使A的平衡度由-1变为-2，以A为根的子树失去平衡。

调整： 提升A的右孩子C的左孩子B为新子树的根结点，A为B的左孩子，C为B的右孩子，将B的左子树CL调整为A的右子树，将B的右子树BR调整为C的左子树，如图所示。

4.2 算法性能分析



采用平衡二叉树提高了查找操作的速度，但是使插入和删除操作复杂化，因此平衡二叉树适用于二叉排序树一经建立就很少进行插入和删除操作而主要进行查找操作的场合中，其查找的时间复杂度为$O(\log_2 n)$。

4.3 B-树和B+树



1.B-树的概念

B-树是一种平衡的多路查找树，在文件系统中B-树已经成为索引文件的一种有效结构。一棵m阶的B-树是满足下列特征的m叉树。

(1) 树中的每个结点最多有m棵子树。

(2) 若根结点不是叶子结点，则至少有两棵子树。

(3) 所有的非终端结点包含信息

$(n,P_0,K_1,P_1,K_2,P_2,\ldots,K_n,P_n)$。其中，$K_i(1\le i\le n)$为关键字，且$K_i<K_i+1$； $P_j(0\le j<n)$是指向子树根结点的指针且P_j所指子树中所有结点的关键字值都小于K_j+1，P_n所指子树中所有结点的关键字值均大于K_n。

4.3 B-树和B+树



2. B+树的结构和差异

B+和B-树的结构大致相同，一颗m阶的B-树和一颗m阶的B+树的差异在于：

（1）在B-树中，每一个结点含有n个关键字和n+1棵子树； 而在B+树中，每一个结点含有n个关键字和n棵子树。

（2）在B-树中，每个结点中的关键字个数n的取值范围是m2-1≤n≤m-1； 而在B+树中，每个结点中的关键字个数n的取值范围是m2≤n≤m，树的根结点的关键字个数的取值范围是1≤n≤m。

（3）B+树中的所有叶子结点包含了全部关键字及指向对应记录的指针，且所有叶子结点按关键字值从小到大的顺序依次链接。

（4）B+树中所有非叶子结点仅起到索引的作用，即结点中的每一个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有该关键字对应记录的存储地址。

4.4 哈希表查找



1.相关概念

哈希存储是以关键字值为自变量通过一定的函数关系(称为散列函数或者哈希函数)计算出数据元素的存储地址，并将该数据元素存入到相应地址的存储单元。在查找时只需要根据查找的关键字采用同样的函数计算出存储地址即可到相应的存储单元取得数据元素。

4.4 哈希表查找



1.相关概念

对于含有n个数据元素的集合总能找到关键字与哈希地址一一对应的函数。若选取函数f(key)=key，数据元素中的最大关键字为m，需要分配m个存储单元，由于关键字集合比存储空间大得多，可能造成存储空间的很大浪费。此外，通过哈希函数变换后可能将不同的关键字映射到同一个哈希地址上，这种现象叫冲突。所以使用哈希方法进行查找时需要关注两个问题，一是要构造好的哈希函数，尽量加快地址计算速度，减少存储空间的浪费； 二是制定解决冲突的方法。

根据哈希函数和处理冲突的方法，将一组关键字映射到一个有限的、地址连续的地址集合空间上，并且数据元素的存储位置由关键字通过哈希函数计算得来，这样的表称为哈希表。

4.5 哈希函数



哈希函数的构造需要遵循以下两个原则：

1.尽可能将关键字均匀地映射到地址集合空间，减少存储空间的浪费。

2.尽可能降低冲突发生的概率。

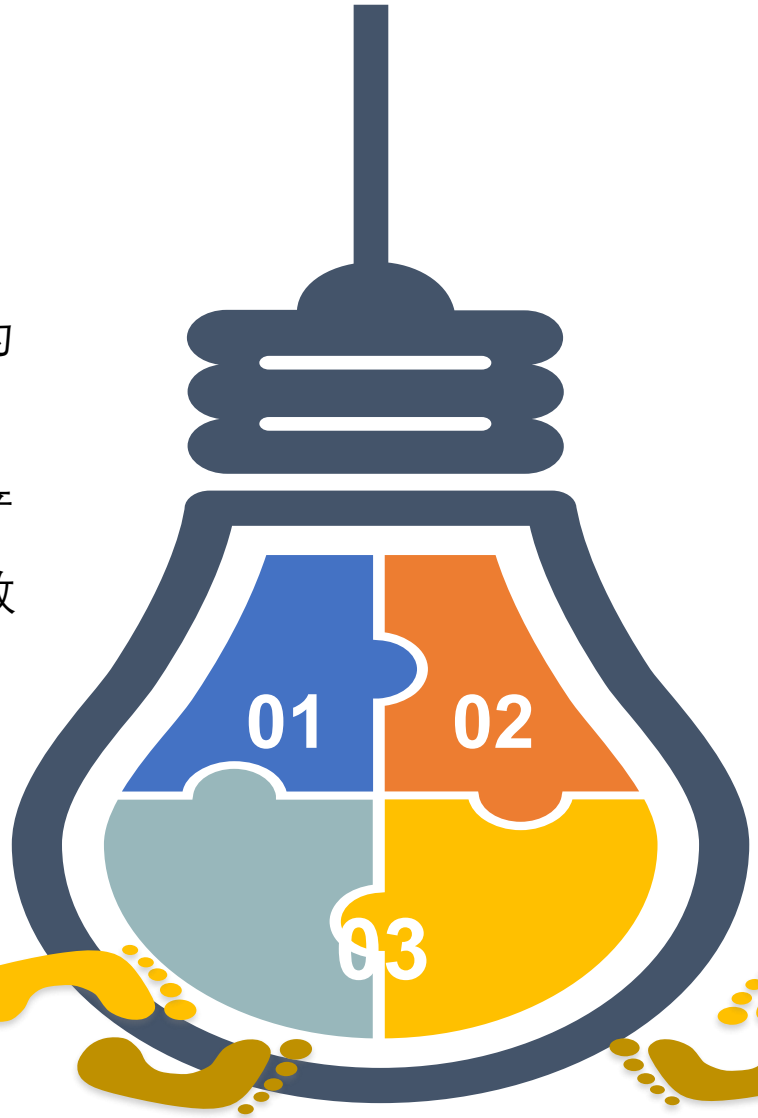
4.5 哈希函数

1.直接地址法

$$H(key) = a * key + b$$

它是取关键字的某个线性函数值为哈希地址。

直接地址法简单，不会产生冲突，但是关键字值往往是离散的，且关键字集合比哈希地址大，会造成存储空间的浪费。



2. 除留余数法

$$H(key) = key\%p(p \le m)$$

它是以关键字除p的余数作为哈希地址，其中m为哈希表长度。

使用除留余数法p的选择很重要，否则会造成严重冲突。例如，若取p=2k，则H(key)=key%p的值仅仅是用二进制表示的key右边的k个位，造成了关键字的映射并不均匀，易造成冲突。通常，为了获得比较均匀的地址分布，一般令p为小于等于m的某个最大素数。

4.5 哈希函数

3.数字分析法

数字分析法是对关键字的各位进行分析，丢掉分布不均匀的位，留下分布均匀的位作为哈希地址。对于不同的关键字集合，所保留的地址可能不相同，因此这种方法主要应用于关键字的位数比存储区域的地址码位数多的情况，并且在使用时需要能预先估计出全体关键字的每一位上各种数字出现的频度的情况。



4.平方取中法

平方取中法是取关键字平方的中间几位作为哈希地址的方法。一个数的平方值的中间几位和数的每一位都有关系，因此平方取中法得到的哈希地址和关键字的每一位都有关系，使得哈希地址的分布较为均匀。

平方取中法适用于关键字中的每一位取值都不够分散或者较分散的位数小于哈希地址所需要的位数的情况。

4.5 哈希函数

5.折叠法

折叠法是将关键字自左向右或自右向左分成位数相同的几部分，最后一部分位数可以不同，然后将这几部分叠加求和，并按哈希表的表长取最后几位作为哈希地址。常用的折叠法有以下两种。

(1) 移位叠加法： 将分割后的各部分的最低位对齐，然后相加。

(2) 间界叠加法： 从一端向另一端沿分割界来回折叠后对齐最后一位相加。

折叠法适用于位数较多，并且每一位的取值都分散均匀的情况。



6.随机数法

随机数法是取关键字的随机数函数值为它的哈希地址，即：

H(key)=random(key)

此方法主要适用于关键字长度不相等的情况。

4.6 解决冲突的方法-开放定址法

开放定址法是当冲突发生时形成一个地址序列，沿着这个地址序列逐个探测，直到找到一个空的开放地址，将发生冲突的数据存放到该地址中。

地址序列的值可表示如下：

$$H_i = (H(key) + d_i)\%m \ (i = 1, 2, ..., k(k \leq m-1))$$

其中H(key)是关键字值为key的哈希函数，m为哈希表长，d_i为每次探测时的地址增量。

根据地址增量取值的不同可以得到不同的开放地址处理冲突探测方法，主要分为3种。



4.6 解决冲突的方法-开放定址法

1.线性探测法：

线性探测法的地址增量如下：

$$d_i = 1, 2, ..., m-1$$

其中i为探测次数。这种方法在解决冲突时，依次探测下一个地址，直到找到一个空的地址，若在整个空间中都找不到空地址将产生溢出。

利用线性探测法解决冲突问题容易造成数据元素的“聚集”，即多个哈希地址不同的关键字争夺同一个后继哈希地址。假设表中的第i、i+1、i+2地址非空，则下一次哈希地址为i、i+1、i+2的数据都企图填入到i+3的位置处。这种现象发生的根本原因是查找序列过分集中在发生冲突的存储单元后面，没有在整个哈希表空间分散开来。



4.6 解决冲突的方法-开放定址法

2.二次探测法：

二次探测法的地址增量如下：

$$d_i = 1^2, -1^2, 2^2, -2^2, \ldots, k^2, -k^2 (k \leq \frac{m}{2})$$

其中m为哈希表长。这种方法能够避免“聚集”现象的发生，但是不能探测到哈希表上的所有存储单元。



4.6 解决冲突的方法-开放定址法

3. 双哈希函数探测法：

双哈希函数探测法是使用另外一个哈希函数RH(key)计算地址增量。哈希地址的计算方法可以表示如下：

$$H_i = (H(key) + i * RH(key))\%m \ (i = 1, 2, ..., m-1)$$

这种方法也可以避免“聚集”现象的发生。

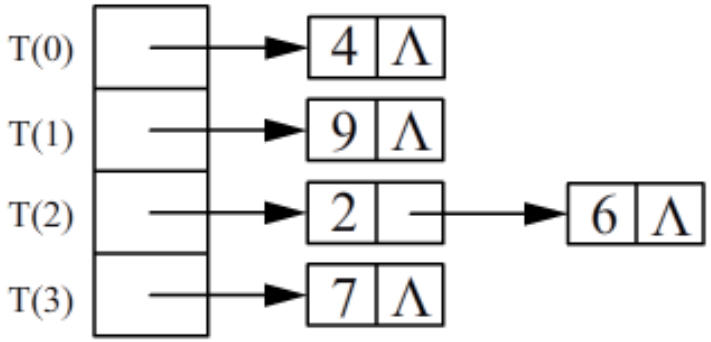


4.6 解决冲突的方法-链地址法



链地址法是将所有具有相同哈希地址的不同关键字的数据元素链接到同一个单链表中。若哈希表的长度为m，则可将哈希表定义为一个由m个头指针组成的指针数组T［0..m-1］，凡是哈希地址为i的数据元素均以结点的形式插入到T［i］为头指针的链表中。

假设一组数据元素的关键字序列为{2,4,6,7,9}，按照哈希函数H(key)=key%4和链地址法处理冲突得到的哈希表如图所示。



4.6 解决冲突的方法-公共溢出区法和再哈希法

公共溢出区法

1.公共溢出区法

公共溢出区法是另建一个溢出表，当不发生冲突时数据元素存入基本表，当发生冲突时数据元素存入溢出表。

2.再哈希法

再哈希法

再哈希法是当发生冲突时再使用另一个哈希函数得到一个新的哈希地址，若再发生冲突，则再使用另一个函数，直到不发生冲突为止。此种方法需要预先设计一个哈希函数序列：

$$H_i = RH_i(key)\ (i = 1, 2, \ldots, k)$$

这种方法不易产生“聚集”现象，但会增加计算的时间。

哈希表查找性能分析

使用平均查找长度来衡量哈希表的查找效率，在查找过程中与关键字的比较次数取决于哈希函数的选取和处理冲突的方法。假设哈希函数是均匀的，即对同样一组随机的关键字出现冲突的可能性是相同的。因此，哈希表的查找效率主要取决于处理冲突的方法。发生冲突的次数和哈希表的装填因子有关，哈希表的装填因子如下：

$$\alpha = \frac{\text{哈希表中的数据元素个数}}{\text{哈希表的长度}}$$

填入表中的数据元素越多，α越大，产生冲突的可能性越大； 填入表中的数据元素越少，α越小，产生冲突的可能性越小。α通常取1和1/2之间的较小的数。



哈希表查找性能分析

处理冲突的方法	平均查找长度	
	查找成功时	查找不成功时
线性探测法	$S_{nl} \approx \frac{1}{2}(1+\frac{1}{1-\alpha})$	$U_{nl} \approx \frac{1}{2}\left[1+\frac{1}{(1-\alpha)^2}\right]$
二次探测法	$S_{nr} \approx -\frac{1}{\alpha}\ln(1-\alpha)$	$U_{nr} \approx \frac{1}{1-\alpha}$
双哈希法	$S_{nr} \approx -\frac{1}{\alpha}\ln(1-\alpha)$	$U_{nr} \approx \frac{1}{1-\alpha}$
链地址法	$S_{nc} \approx 1+\frac{\alpha}{2}$	$U_{nc} \approx \alpha + e^{-\alpha}$



小结

(1) 查找就是在由一组记录组成的集合中寻找属性值符合特定条件的数据元素。若集合中存在符合条件的记录，则查找成功，否则查找失败。

(2) 查找表是一种以同一类型的记录构成的集合为逻辑结构、以查找为核心运算的灵活的数据结构。在实现查找表时要根据实际情况按照查找的具体要求组织查找表，从而实现高效率的查找。

(3) 静态查找表是指对表的操作不包括对表的修改的表，可以用顺序表或线性链表进行表示，分为顺序查找、二分查找和分块查找3种。



小结

(4) 动态查找表是指对表的操作包括对表的修改的表，即表结构本身实际是在查找过程中动态生成的。动态查找表有多种不同的实现方法，如二叉排序树查找。

(5) 平衡二叉树是左、右子树深度之差的绝对值小于2并且左、右子树均为平衡二叉树的树。平衡二叉树又叫AVL树。

(6) 哈希存储以关键字值为自变量，通过哈希函数计算出数据元素的存储地址，并将该数据元素存入到相应地址的存储单元。在进行哈希表查找时只需要根据查找的关键字采用同样的函数计算出存储地址即可到相应的存储单元取得数据元素。在进行哈希表查找时需要构造好的哈希函数并且制定解决冲突的方法。



谢谢观看